

Project: Resale app finder

User:

Users want an easier way to be able to find specific clothing, brands, and styles being sold on resale marketplaces.

Goal:

Let the user search for a specific item, brand, or style and save listings they are interested in so they can easily compare and revisit them.

MVP Behaviors:

1. Search form with search term, brand, price, and optional size.
2. Search terms are required, brand, price, and size are optional.
3. Submitting a search gives matching resale listings.
4. Each listing shows the item name, brand, price, size, image, marketplace, and a link to the original listing.
5. Allow the user to save/favorite listings.
6. Show saved listings on a separate Saved page or section.
7. Saved listings remain after refreshing or reopening the deployed web app.
8. Unsaved listening, but visited, show in a recently viewed section, only 10 most recently visited listings.
9. Allow users to remove listings from Saved.
10. Show a clear loading state while search results or saved listings are loading.
11. Show a useful error message if searching, loading, or saving fails.

ACCEPTANCE CRITERIA

1. When I open the app, I can search for an item or brand.
2. When I search "Chrome Hearts hoodie," relevant listings appear.
3. When I add filters such as Brand = Chrome Hearts, Size = Medium, and Maximum Price = \$300, the displayed results match those filters.
4. A blank search term is rejected.
5. A negative or invalid maximum price is rejected.
6. Each result clearly shows which marketplace the item comes from.
7. Clicking a listing gives me a way to view the original marketplace listing.
8. When I save a listing, it appears in my Saved section.
9. After refreshing the page, my saved listings are still there.
10. When I remove a saved item, it disappears from Saved.
11. If the backend returns an error, the page does not pretend that the search or save succeeded.
12. The layout is usable at laptop and phone browser widths.

Constraints:

CONSTRAINTS

- Next.js App Router.
- Frontend communicates with a Next.js Route Handler backend.
- Persistent storage uses the class Supabase workflow.
- Database credentials that provide elevated access must remain server-side.
- Use only non-sensitive practice data.

NON-GOALS

No login/accounts, sharing, payments, notifications, external APIs, AI features, mobile-native app, or analytics.